

## Section 5.3: Probabilistic Data Structures

---

*Study Guide | Part 3 of 4 | System Design Interview Prep Series*

**Note on math-heavy content:** This section covers structures with non-trivial error bounds. For each structure, worked numeric examples and proof-by-contradiction style arguments for correctness are provided to build deep intuition — not just formula recall.

---

### Executive Overview

Probabilistic data structures trade a small, bounded probability of error for dramatic reductions in memory and time complexity. The key interview insight: **knowing when exact answers are unnecessary** is as important as knowing the algorithm.

Three structures dominate system design interviews: - **Bloom Filter**: “Does this element exist?” (no false negatives, bounded false positives) - **HyperLogLog (HLL)**: “How many distinct elements are there?” ( $\pm 1-2\%$  error, constant memory) - **Count-Min Sketch (CMS)**: “How often does this element appear?” (overcount-bounded error)

---

### Bloom Filters

#### How It Works — With Full Intuition

A Bloom filter is a bit array of  $m$  bits, all initialized to 0, plus  $k$  independent hash functions.

**Add element**  $x$ :

```
For each hash function  $h_i$  ( $i = 1..k$ ):  
  Compute position  $p_i = h_i(x) \bmod m$   
  Set  $bits[p_i] = 1$ 
```

**Query element**  $y$ :

For each hash function  $h_i$ :

If  $\text{bits}[h_i(y) \bmod m] == 0 \rightarrow \text{DEFINITELY NOT in set (no false negative)}$

If ALL bits are 1  $\rightarrow \text{PROBABLY in set (false positive possible)}$

**Why no false negatives?** When you add  $x$ , you set specific bit positions. They can never be unset (no deletions in basic Bloom filter). If  $x$  was added, all its bit positions are 1. So querying  $x$  will always find all bits set. This is a correctness argument by construction: false negatives are structurally impossible.

**Why false positives?** Another element  $z$  (not in the set) might coincidentally hash to bit positions that were all set by *other* elements. The filter sees all 1s and incorrectly says “probably present.”

---

## False Positive Rate — Worked Numeric Example

### Formula:

After inserting  $n$  elements into  $m$  bits with  $k$  hash functions:

$$P(\text{a specific bit is still } 0) = (1 - 1/m)^{(kn)} \approx e^{(-kn/m)}$$

False positive rate:

$$\epsilon = (1 - e^{(-kn/m)})^k$$

Optimal  $k$  (minimizes  $\epsilon$  for given  $m, n$ ):

$$k^* = (m/n) \times \ln(2) \approx 0.693 \times (m/n)$$

### Worked example — 1M users, 1% FP rate:

Target:  $n = 1,000,000$  elements,  $\epsilon = 0.01$

Step 1: Compute  $m$  (bits needed)

$$\begin{aligned} m &= -n \times \ln(\epsilon) / (\ln 2)^2 \\ &= -1,000,000 \times \ln(0.01) / (0.693)^2 \\ &= -1,000,000 \times (-4.605) / 0.480 \\ &= 9,590,000 \text{ bits} \approx 9.6 \text{ Mbits} = 1.2 \text{ MB} \end{aligned}$$

Step 2: Compute optimal  $k$

$$k^* = (m/n) \times \ln(2) = 9.6 \times 0.693 \approx 6.6 \rightarrow \text{round to 7 hash functions}$$

Step 3: Verify false positive rate with  $k=7$

$$\begin{aligned} \epsilon &= (1 - e^{(-7 \times 1M / 9.6M)})^7 \\ &= (1 - e^{(-0.729)})^7 \\ &= (1 - 0.482)^7 \\ &= (0.518)^7 \\ &\approx 0.009 = 0.9\% \checkmark \text{ (under 1\%)} \end{aligned}$$

Memory comparison:

Bloom filter: 1.2 MB for 1M elements  
Hash set: ~50 MB (avg 50 bytes/string × 1M)  
Savings: ~40x

### Proof by contradiction that false negatives are impossible:

Assume: Element  $x$  was added to the filter, but a query for  $x$  returns "not present."  
→ This means at least one bit position  $h_i(x)$  is 0.  
→ But when  $x$  was added, we SET  $\text{bits}[h_i(x)] = 1$  for all  $i$ .  
→ Bits can only be set to 1, never reset to 0 (no deletion).  
→ Contradiction: the bit cannot be 0 if  $x$  was added.  
∴ False negatives are impossible. □

## Sizing Cheat Sheet

| Elements (n) | Target FP Rate | Bits (m) | Memory | Hash fns (k) |
|--------------|----------------|----------|--------|--------------|
| 1M           | 1%             | 9.6M     | 1.2 MB | 7            |
| 1M           | 0.1%           | 14.4M    | 1.8 MB | 10           |
| 10M          | 1%             | 96M      | 12 MB  | 7            |
| 1B           | 1%             | 9.6B     | 1.2 GB | 7            |

**Rule of thumb:** ~10 bits per element gives ~1% FP rate with ~7 hash functions.

## Production Applications

**LSM-Tree SSTable skipping** (used in LevelDB, RocksDB, Cassandra):

Problem: LSM compaction creates many SSTables (immutable sorted files).  
A read for key  $K$  must check every SSTable – expensive.

Solution: Bloom filter per SSTable  
Before reading SSTable from disk → check Bloom filter  
If filter says "definitely not here" → skip SSTable entirely  
Typical result: 95%+ of SSTable reads skipped for cold keys

**Cache penetration defense** (Section 5.2 cross-reference):

```
# On startup
bf = redis.bf_reserve("known_users", 0.01, 10_000_000) # 1% FP, 10M cap
for uid in db.stream_all_user_ids():
    redis.bf_add("known_users", uid)

# On request
def get_user(user_id: int):
    if not redis.bf_exists("known_users", user_id):
        return None # Definitely not a user — skip DB
    return cache_or_db_lookup(user_id)
```

## Counting Bloom Filters (Extension for Deletions)

Standard Bloom filters don't support deletion (unsetting a bit might affect other elements that hash there). Counting Bloom Filters replace each bit with a small counter (typically 4 bits = max count 15).

```
Add x:    Increment counters at h1(x), h2(x), ..., hk(x)
Remove x:  Decrement counters at h1(x), h2(x), ..., hk(x)
Query x:   If any counter == 0 → definitely not present

Risk: Counter overflow at 15. If an element appears >15 times,
      counter saturates. Solution: Use 8-bit counters (cap at 255).

Memory cost: 4x-8x vs standard Bloom filter (4 or 8 bits per position vs 1 bit)
```

## HyperLogLog (HLL)

### The Core Insight

**Problem:** Count distinct elements in a stream of billions (cardinality estimation).

**Naive approach:** Put everything in a hash set. Cost:  $O(n)$  memory = gigabytes for billions.

**HLL insight:** The maximum number of leading zeros in a random hash value tells you (statistically) how many distinct elements you've seen.

**Intuition with coin flips:**

```
If you flip a fair coin until you get heads:
Getting 1 flip (T=0 leading heads) → very common, probably small n
Getting 5 flips (TTTTH) → less common, implies larger n
Getting 20 flips (19 T's then H) → very rare, implies huge n
```

HLL hashes each element and counts leading zeros in the hash.  
The maximum leading zeros seen so far estimates  $\log_2(n)$ .

## HLL Algorithm — Numeric Walkthrough

**Setup:** Maintain  $m = 2^b$  registers (typically  $m = 4096$ ,  $b = 12$ ).

For each incoming element  $x$ :

1. Hash  $x$  to a 64-bit value:  $h = \text{hash}(x)$
2. Use first  $b$  bits of  $h$  to select register index  $j$
3. Use remaining  $64-b$  bits to count leading zeros:  $p = \text{count\_leading\_zeros}(h[b:]) + 1$
4. Update:  $\text{register}[j] = \max(\text{register}[j], p)$

Estimate:

$\text{cardinality} \approx \alpha_m \times m^2 \times (\sum 2^{-\text{register}[j]})^{-1}$   
(harmonic mean of  $2^{\text{register}}$  values, scaled by correction constant  $\alpha_m$ )

### Worked example — small trace:

$m = 8$  registers ( $b=3$ ), all initialized to 0

Stream: [user\_1, user\_2, user\_3, user\_1 (duplicate)]

hash(user\_1) = 001|00111... → register 1,  $p = 3$  → register[1] =  $\max(0, 3) = 3$   
hash(user\_2) = 101|10000... → register 5,  $p = 1$  → register[5] =  $\max(0, 1) = 1$   
hash(user\_3) = 001|11000... → register 1,  $p = 1$  → register[1] =  $\max(3, 1) = 3$  (no change)  
hash(user\_1) = 001|00111... → register 1,  $p = 3$  → register[1] =  $\max(3, 3) = 3$  (duplicate handled)

registers: [0, 3, 0, 0, 0, 1, 0, 0]

Harmonic mean estimate → ~3 distinct elements ✓

## Error Bounds and Memory

Standard error:  $\sigma = 1.04 / \sqrt{m}$

For  $m = 4096$  registers (standard):

$\sigma = 1.04 / \sqrt{4096} = 1.04 / 64 \approx 0.016 = 1.6\%$  error

Memory: 4096 registers  $\times$  5 bits each = 20,480 bits  $\approx$  2.5 KB

Compare to exact counting:

1B distinct users  $\times$  8 bytes (int64) = 8 GB

HLL: 2.5 KB for same workload. Memory reduction: 3,200,000x

### Proof that HLL underestimates for small cardinalities (why bias correction matters):

For  $n \ll m$ : Most registers are 0.  
The harmonic mean of  $2^{-0} = 1$  for all zero registers pulls the estimate toward the lower end.

Fix: "Linear counting" for  $n < m \times 2.5$   
count non-zero registers:  $V$   
estimate =  $m \times \ln(m/V)$

For  $n > 2^{32}$  (very large): Separate correction for hash collision range.

## Redis HLL Operations

```
-- Add elements (streaming)
PFADD visitors:2024-06-07 user_1 user_2 user_3

-- Count (estimate)
PFCOUNT visitors:2024-06-07
→ 3 (in this case exact; at scale, ±1.6%)

-- Merge multiple HLLs (union – useful for weekly/monthly rollups)
PFMERGE visitors:2024-06 visitors:2024-06-07 visitors:2024-06-06 ...
PFCOUNT visitors:2024-06
→ Monthly unique visitors
```

**Key property:** `PFMERGE` gives the **union** cardinality, not the sum. This is mathematically correct for “how many distinct users this month” across daily HLLs.

## Count-Min Sketch (CMS)

### Structure and Algorithm

**Problem:** Estimate the frequency of elements in a stream, using sub-linear memory.

Structure:  $d$  rows  $\times$   $w$  columns matrix of integer counters

Parameters:

- $w = \lceil \frac{re}{\epsilon} \rceil$  (width, controls accuracy)
- $d = \lceil \ln(1/\delta) \rceil$  (depth, controls confidence)

$\epsilon$  = desired accuracy (e.g., 0.05 = 5% of total count)  
 $\delta$  = failure probability (e.g., 0.01 = 1% chance of exceeding error)

Add element  $x$ :

For each row  $i$  (1..d): increment  $\text{counter}[i][h_i(x) \bmod w]$

Estimate frequency of  $x$ :

Return  $\min(\text{counter}[i][h_i(x)] \text{ for } i \text{ in } 1..d)$

---

## Worked Numeric Example

**Setup: Track top URLs for rate limiting, 1M requests, 5% accuracy, 1% failure:**

$\epsilon = 0.05, \delta = 0.01$

$w = \lceil re / 0.05 \rceil = \lceil 2.718 / 0.05 \rceil = \lceil 54.4 \rceil = 55$  columns

$d = \lceil \ln(1/0.01) \rceil = \lceil \ln(100) \rceil = \lceil 4.605 \rceil = 5$  rows

Memory:  $55 \times 5 \times 4 \text{ bytes} = 1,100 \text{ bytes} \approx 1.1 \text{ KB}$  (for 1M item stream!)

Error guarantee:

True count of  $x = f(x)$

CMS estimate  $\leq f(x) + \epsilon \times N$  where  $N$  = total count

If  $N = 1\text{M}$  and  $\epsilon = 0.05$ :

Estimate  $\leq \text{true\_count} + 50,000$

For a URL with 500,000 true requests:

CMS might return up to 550,000 – still identifies it as a heavy hitter

**Why min and not average?** Each row's counter can only *overcount* (hash collisions add extra), never undercount. Taking the minimum across all rows gives the tightest (lowest) overestimate.

**Proof CMS never undercounts:**

For row  $i$ ,  $\text{counter}[i][h_i(x)]$  counts  $x$ 's true occurrences PLUS any other element  $y$  where  $h_i(y) = h_i(x)$  (collision).

Each collision only adds to the count, never subtracts.

$\therefore \text{counter}[i][h_i(x)] \geq \text{true\_count}(x)$  for every row  $i$ .

$\therefore \min \text{ across rows } \geq \text{true\_count}(x)$ . No undercounting.  $\square$

## Heavy Hitter Detection

```
class CountMinSketch:
    def __init__(self, epsilon=0.05, delta=0.01):
        import math, hashlib
        self.w = math.ceil(math.e / epsilon)
        self.d = math.ceil(math.log(1 / delta))
        self.table = [[0] * self.w for _ in range(self.d)]
        self.total = 0

    def add(self, item: str, count: int = 1):
        self.total += count
        for i in range(self.d):
            j = hash((item, i)) % self.w
            self.table[i][j] += count

    def estimate(self, item: str) -> int:
        return min(
            self.table[i][hash((item, i)) % self.w]
            for i in range(self.d)
        )

    def is_heavy_hitter(self, item: str, threshold: float = 0.01) -> bool:
        """Is this item in the top `threshold` fraction of all traffic?"""
        return self.estimate(item) > self.total * threshold
```

---

## Production Applications

### Real-time rate limiting:

```
cms = CountMinSketch(epsilon=0.01, delta=0.001)

def check_rate_limit(ip: str, limit: int = 1000) -> bool:
    cms.add(ip)
    current_rate = cms.estimate(ip)

    if current_rate > limit:
        return False    # Rate limited
    return True

# Memory cost: ~12 KB regardless of number of distinct IPs
# vs. storing exact counts for 1M IPs: ~50 MB
```

### DDoS source detection:

```
Network monitoring at 10 Gbps line rate:
Exact per-IP counting: ~50 MB hash map, too slow for hardware
```



CMS: 12 KB, 5 hash lookups per packet  
Identifies IPs sending > 1% of traffic with 99% confidence

## Comparison: When to Use Each Structure

| Question                              | Structure        | Error Type                        | Memory           | Use Case                         |
|---------------------------------------|------------------|-----------------------------------|------------------|----------------------------------|
| "Does X exist?"                       | Bloom Filter     | False positives only              | ~10 bits/element | Dedup, cache penetration defense |
| "How many distinct?"                  | HyperLogLog      | $\pm 1-2\%$ count                 | ~2.5 KB constant | Analytics, DAU counting          |
| "How often does X appear?"            | Count-Min Sketch | Overcounts by $\epsilon \times N$ | ~1–10 KB         | Rate limiting, top-K             |
| "Exact count of X"                    | Hash Map         | None                              | $O(n)$           | Use only for small n             |
| "Does X appear often?" (with deletes) | Counting Bloom   | FP + counter overflow             | ~4× Bloom        | Rate limiting with resets        |

## Interview Question 1 — Unique Visitor Counting at Scale

**Prompt:** Design a web analytics system that counts unique visitors to a website with 1B daily visitors. How do you achieve sub-linear memory usage?

### Solution: HyperLogLog per Day + PFMERGE for Rollups

```
# Record a visit
def record_visit(user_id: str):
    today = date.today().isoformat() # e.g., "2024-06-07"
    redis.pfadd(f"visitors:{today}", user_id)
    redis.expire(f"visitors:{today}", 32 * 86400) # Keep 32 days

# Daily unique count
def daily_uniques(date_str: str) -> int:
    return redis.pfcount(f"visitors:{date_str}")

# Weekly unique count (not sum of dailies – users visit multiple days)
def weekly_uniques(year: int, week: int) -> int:
```

```

from datetime import date, timedelta
start = date.fromisocalendar(year, week, 1)
keys = [f"visitors:{{(start + timedelta(days=i)).isoformat()}}" for i in
range(7)]

temp_key = f"visitors:temp:week:{{year}}:{{week}}"
redis.pfmerge(temp_key, *keys)
redis.expire(temp_key, 3600) # Cleanup after 1 hour
return redis.pfcount(temp_key)

```

### Memory:

Daily HLL: 2.5 KB per day  
 30 days of history:  $30 \times 2.5 \text{ KB} = 75 \text{ KB}$  total

vs. exact counting (hash set):  
 $1\text{B user IDs} \times 8 \text{ bytes} = 8 \text{ GB per day}$

HLL: 75 KB for 30 days. Exact: 240 GB. Reduction: 3.2M×

### Error analysis:

HLL error:  $\pm 1.6\%$  for 4096 registers  
 At 1B visitors:  $\pm 16\text{M}$  visitors – acceptable for analytics  
 Not acceptable for billing (use exact counting for charged metrics)

---

## Interview Question 2 — Top-K Heavy Hitters in Real-Time Traffic

---

**Prompt:** Detect the top 100 most-requested API endpoints from a stream of 500K requests/second, using <10 MB memory.

### Solution: Count-Min Sketch + Min-Heap

```

import heapq
from collections import defaultdict

cms = CountMinSketch(epsilon=0.001, delta=0.001) # ~25 KB
top_k_heap = [] # Min-heap of (count, endpoint)
top_k_size = 100
seen = set()

def process_request(endpoint: str):
    cms.add(endpoint)
    estimate = cms.estimate(endpoint)

```

```

if endpoint not in seen:
    seen.add(endpoint)
    if len(top_k_heap) < top_k_size:
        heapq.heappush(top_k_heap, (estimate, endpoint))
    elif estimate > top_k_heap[0][0]:
        heapq.heapreplace(top_k_heap, (estimate, endpoint))

# Periodically resync heap estimates with CMS
# (every N requests, update all heap entries with fresh CMS estimates)

def get_top_k():
    return sorted(top_k_heap, reverse=True)

```

### Memory breakdown:

```

CMS: ~25 KB
Heap (100 entries): ~10 KB
Seen set (up to 10K distinct endpoints): ~500 KB
Total: ~535 KB << 10 MB target

```

## Interview Question 3 — Distributed Bloom Filter for URL Deduplication in a Web Crawler

**Prompt:** Design a distributed web crawler that avoids re-crawling URLs. You have 10B URLs total. How do you track which URLs have been crawled?

### Solution: Distributed Bloom Filter in Redis

```

Setup:
n = 10B URLs
Target FP rate  $\epsilon$  = 0.001 (0.1%)

 $m = -10B \times \ln(0.001) / (\ln 2)^2$ 
    =  $-10B \times (-6.908) / 0.480$ 
    = 143.9B bits  $\approx$  17.5 GB

 $k = 0.693 \times (m/n) = 0.693 \times 14.4 \approx 10$  hash functions

Distribution: Shard the 17.5 GB bit array across 4 Redis nodes
Each node holds ~4.4 GB shard
Hash functions map to shard + position

```

```

class DistributedBloomFilter:
    def __init__(self, redis_nodes, m, k):
        self.nodes = redis_nodes

```

```

self.m = m
self.k = k

def _positions(self, url: str):
    import mmh3
    return [mmh3.hash(url, seed=i) % self.m for i in range(self.k)]

def add(self, url: str):
    for pos in self._positions(url):
        node = self.nodes[pos % len(self.nodes)]
        node.setbit("bloom", pos // len(self.nodes), 1)

def might_contain(self, url: str) -> bool:
    for pos in self._positions(url):
        node = self.nodes[pos % len(self.nodes)]
        if not node.getbit("bloom", pos // len(self.nodes)):
            return False    # Definitely not crawled
    return True    # Probably crawled

```

### Trade-off analysis:

FP rate 0.1%: 1 in 1000 URLs mistakenly skipped  
 At 10B URLs: ~10M missed URLs – acceptable for large-scale crawl

Alternative: Exact set (hash map)  
 10B URLs × avg 50 bytes = 500 GB RAM – prohibitively expensive

Bloom filter wins: 17.5 GB vs 500 GB (29× reduction) at 0.1% FP cost